# Vídeo 6.1: Por que testar e seu primeiro teste automatizado

**Duração-alvo:** 6 min | **Conceito:** teste automatizado | **O aluno sai sabendo:** o que é um teste automatizado e como rodar o primeiro.

**Antes de gravar:** partir do código do fim do módulo 5 (versão v5). Os testes usam o Jest e o Supertest. Conferir o comando `npm test` no computador de gravação.

**[0:00] Abertura**

*Na tela:* título do vídeo.

*Fala:* “Você construiu uma API completa. Mas como ter certeza de que ela continua funcionando depois que você mudar alguma coisa? Testando cada rota à mão, toda vez? Isso cansa, e a gente esquece. Neste módulo vamos escrever testes automatizados: pequenos programas que testam a API por nós.”

**[0:50] A lista de conferência**

*Na tela:* uma lista de conferência de fechamento da biblioteca: portas trancadas, luzes apagadas, alarme ligado.

*Fala:* “Pense em fechar a biblioteca à noite. A funcionária segue uma lista: portas trancadas? Luzes apagadas? Alarme ligado? Ninguém confia só na memória. Um teste automatizado é uma lista de conferência que o computador executa. Cada item é um teste: faz um pedido à API e confere se a resposta é a esperada. Se algo quebrar, o teste avisa na hora, antes de qualquer cliente descobrir.”

**[1:50] Instalando as ferramentas**

*Na tela:* no terminal, digitar o comando abaixo; depois, no `package.json`, acrescentar o script de teste.

```
npm install --save-dev jest supertest
```

```
"scripts": {
  "test": "jest"
}
```

*Fala:* “Duas ferramentas. O Jest executa os testes e mostra o resultado. O Supertest faz pedidos à nossa API de dentro do teste, sem precisar ligar o servidor. É por isso que separamos o app do server no vídeo anterior. O traço traço save-dev avisa que são ferramentas só para desenvolver, e não fazem parte da API em si. No package ponto json, na parte de scripts, escrevo: test, dois pontos, jest. Assim, o comando npm test roda os testes.”

**[3:00] Fechando o banco ao terminar**

*Na tela:* no `src/db.js`, acrescentar a função abaixo e exportá-la.

```
async function fecharBanco() {
  if (conexao) {
        await conexao.close();
        conexao = undefined;
  }
}

 module.exports = { obterBanco, fecharBanco };
```

*Fala:* “Uma pequena adição no db ponto js: uma função para fechar o banco. Sem ela, o Jest termina os testes, mas o programa não encerra, porque o banco continua aberto, como a porta destrancada da lista de conferência.”

**[3:40] O primeiro teste**

*Na tela:* criar o arquivo tests/status.test.js, com o código abaixo.

```
process.env.ARQUIVO_BANCO = ':memory:';

 const request = require('supertest');
 const app = require('../src/app');
 const { fecharBanco } = require('../src/db');

 afterAll(async () => {
  await fecharBanco();
 });

 describe('GET /status', () => {
  test('responde 200 e informa que a API está funcionando', async () => {
        const resposta = await request(app).get('/status');

        expect(resposta.status).toBe(200);
        expect(resposta.body.ok).toBe(true);
  });
 });
```

*Fala:* “A primeira linha do teste diz que, durante os testes, o banco será dois pontos memory dois pontos: um banco temporário, que fica na memória e desaparece quando o teste acaba. Assim, nunca mexemos nos livros de verdade. Depois importamos o Supertest, o app e a função de fechar o banco. O describe agrupa testes sobre o mesmo assunto. O test descreve, com uma frase, o que deve acontecer. Dentro dele, fazemos o pedido: request, app, get, barra status, com await, porque é assíncrono. E depois as conferências: expect resposta ponto status toBe 200, ou seja, espera-se que o status seja 200. E que o campo ok seja true. O afterAll roda no final e fecha o banco.”

**[4:40] Rodando e quebrando de propósito**

*Na tela:* rodar npm test e mostrar o resultado verde. Depois trocar ok: true por ok: false na rota de status do src/app.js, rodar de novo e mostrar a falha. Desfazer a mudança.

*Fala:* “Rodo npm test. Verde: PASS, um teste passou. Agora vamos quebrar de propósito. Troco ok true por ok false no app. Rodo de novo: FAIL. O Jest mostra o que era esperado,

true, e o que recebeu, false, e aponta a linha. É para isso que serve: avisar quando uma mudança estraga o que funcionava. Desfaço a mudança, e o teste volta a passar.”

**[5:40] Resumo**

*Na tela:* três frases curtas.

*Fala:* “Resumindo: um teste automatizado é uma lista de conferência que o computador executa. Instalamos o Jest e o Supertest, escrevemos o primeiro teste e vimos o que acontece quando algo quebra. No próximo vídeo vamos testar também o que dá errado.”

**Exercício sugerido:** acrescentar ao teste uma conferência de que a mensagem de `/status` é a esperada.